Student Name - Hariprasath Mahalingam

System - Night Watch interrupt driven Sense Think Act node (Arduino Uno) Tinkercad Link

https://www.tinkercad.com/things/j8TDZwMCI76-complete-interrupt-driven-system-with-timer-logic/editel?returnTo=https%3A%2F%2Fwww.tinkercad.com%2Fdashboard%2Fdesigns%2Fcircuits&sharecode=gwb_i9ohWolVz67XCwW95YGNcsEPPYg9SdHqWBUKsaM

Video Link - https://youtu.be/1m6_7hwXTwM

Github Link - https://github.com/hariprasath299/night-watch.git

System architecture and logic

I have used a PIR sensor on D2, a button on D8,a tilt switch on D9, and an LDR on A0, driving two LEDs and a buzzer in Arduino Uno to monitor a room in case of movement at night and activation of the tilt sensor.

Every pass of loop() runs three stages in order

- readInputs() copies the interrupt flags.
- decideNextState() applies the rule.
- riveActuators() sets the outputs.

It is a state machine with three states

- Disarmed
- Armed
- Alarm

The alarm requires two conditions at once

- Armed and motion in dark
- Armed and Tilting the switch as light is irrelevant

Interrupt configuration and usage

The PIR uses attachInterrupt() on INT0, RISING and Its ISR only sets a flag

D8 and D9 are PB0 and PB1, so both are served by the single PCINT0_vect vector. I enabled PORT B with PCICR |= (1 << PCIE0) and unmasked both pins in PCMSK0. Since one vector covers two pins and fires on every edge, the ISR reads PINB and XORs it against the previous snapshot to find which bit changed.

Timer1 runs in CTC mode with a 256 prescaler and OCR1A = 6249, giving 10 Hz. The ISR writes the LED patterns directly and defers slower work via flags.

Challenges encountered and how they were resolved

The challenges i have faced in making this system was connecting the right sensors to the Arduino Uno as it took me a lot of time to connect the sensors perfectly to the Arduino Uno as it was one of my challenges

The second was harder because nothing looked broken. Everything compiled and all three interrupts fired, but the trace showed light=6 on every status line regardless of simulated light level. I had fitted a 220 Ω resistor in the LDR divider instead of 10 kΩ. Replacing it restored the full range, 54 to 974.

Key findings and test

I ran three test conditions and compared the response each time.

Test 1: motion while armed, bright vs dark. Motion at light=974 printed "motion ignored, room is bright", while the same input at light=54 raised the alarm. The motion counter rose both times, so the interrupt fired and the think stage made the decision. The response changed because the alarm needs both conditions.

Test 2: event traffic at 10 Hz. Through motion, button and tamper events, status lines stayed 1999 to 2000 ms apart (7999 to 19997 ms), so events never displaced the timer. The 1 ms differences come from comparing millis(), which Timer0 updates in 1.024 ms steps, against Timer1's ticks.

Test 3: timer at 20 Hz. I changed only OCR1A, from 6249 to 3124. Status lines moved to 999 to 1001 ms apart, since the heartbeat is still 20 ticks but each tick is now 50 ms. Motion in the dark while armed still went straight to ALARM at 10650 ms, and the next status line at 10996 ms stayed on the one-second grid. The periodic output changed while the event response did not.